

Chapter 3

An Introduction to the Tools

Man is a tool-using animal. Without tools he is nothing, with tools he is all.

—Thomas Carlyle

Stop Me If You've Heard This One Before

This book is about the techniques of using tests to guide the development of object-oriented software, not about specific technologies. To demonstrate the techniques in action, however, we've had to pick *some* technologies for our example code. For the rest of the book we're going to use Java, with the JUnit 4, Hamcrest, and jMock2 frameworks. If you're using something else, we hope we've been clear enough so that you can apply these ideas in your environment.

In this chapter we briefly describe the programming interfaces for these three frameworks, just enough to help you make sense of the code examples in the rest of the book. If you already know how to use them, you can skip this chapter.

A Minimal Introduction to JUnit 4

We use JUnit 4 (version 4.6 at the time of writing) as our Java test framework.¹ In essence, JUnit uses reflection to walk the structure of a class and run whatever it can find in that class that represents a test. For example, here's a test that exercises a `Catalog` class which manages a collection of `Entry` objects:

```
public class CatalogTest {
    private final Catalog catalog = new Catalog();
    @Test public void containsAnAddedEntry() {
        Entry entry = new Entry("fish", "chips");
        catalog.add(entry);
        assertTrue(catalog.contains(entry));
    }
    @Test public void indexesEntriesByName() {
        Entry entry = new Entry("fish", "chips");
        catalog.add(entry);
        assertEquals(entry, catalog.entryFor("fish"));
        assertNull(catalog.entryFor("missing name"));
    }
}
```

1. JUnit is bundled with many Java IDEs and is available at www.junit.org.

Test Cases

JUnit treats any method annotated with `@Test` as a test case; test methods must have neither a return value nor parameters. In this case, `CatalogTest` defines two tests, called `containsAnAddedEntry()` and `indexesEntriesByName()`.

To run a test, JUnit creates a new instance of the test class and calls the relevant test method. Creating a new test object each time ensures that the tests are isolated from each other, because the test object's fields are replaced before each test. This means that a test is free to change the contents of any of the test object fields.



NUnit Behaves Differently from JUnit

Those working in .Net should note that NUnit reuses the same instance of the test object for all the test methods, so any values that might change must either be reset in `[Setup]` and `[TearDown]` methods (if they're fields) or made local to the test method.

Assertions

A JUnit test invokes the object under test and then makes *assertions* about the results, usually using assertion methods defined by JUnit which generate useful error messages when they fail.

`CatalogTest`, for example, uses three of JUnit's assertions: `assertTrue()` asserts that an expression is true; `assertNull()` asserts that an object reference is null; and `assertEquals()` asserts that two values are equal. When it fails, `assertEquals()` reports the expected and actual values that were compared.

Expecting Exceptions

The `@Test` annotation supports an optional parameter `expected` that declares that the test case should throw an exception. The test fails if it does *not* throw an exception or if it throws an exception of a different type.

For example, the following test checks that a `Catalog` throws an `IllegalArgumentException` when two entries are added with the same name:

```
@Test(expected=IllegalArgumentException.class)
public void cannotAddTwoEntriesWithTheSameName() {
    catalog.add(new Entry("fish", "chips"));
    catalog.add(new Entry("fish", "peas"));
}
```

Test Fixtures

A *test fixture* is the fixed state that exists at the start of a test. A test fixture ensures that a test is repeatable—every time a test is run it starts in the same state so it should produce the same results. A fixture may be *set up* before the test runs and *torn down* after it has finished.

The fixture for a JUnit test is managed by the class that defines the test and is stored in the object's fields. All tests defined in the same class start with an identical fixture and may modify that fixture as they run. For `CatalogTest`, the fixture is the empty `Catalog` object held in its `catalog` field.

The fixture is usually set up by field initializers. It can also be set up by the constructor of the test class or instance initializer blocks. JUnit also lets you identify methods that set up and tear down the fixture with annotations. JUnit will run all methods annotated with `@Before` before running the tests, to set up the fixture, and those annotated by `@After` after it has run the test, to tear down the fixture. Many JUnit tests do not need to explicitly tear down the fixture because it is enough to let the JVM garbage collect any objects created when it was set up.

For example, all the tests in `CatalogTest` initialize the `catalog` with the same entry. This common initialization can be moved into a field initializer and `@Before` method:

```
public class CatalogTest {
    final Catalog catalog = new Catalog();
    final Entry entry = new Entry("fish", "chips");

    @Before public void fillTheCatalog() {
        catalog.add(entry);
    }

    @Test public void containsAnAddedEntry() {
        assertTrue(catalog.contains(entry));
    }

    @Test public void indexesEntriesByName() {
        assertEquals(equalTo(entry), catalog.entryFor("fish"));
        assertNull(catalog.entryFor("missing name"));
    }

    @Test(expected=IllegalArgumentException.class)
    public void cannotAddTwoEntriesWithTheSameName() {
        catalog.add(new Entry("fish", "peas"));
    }
}
```

Test Runners

The way JUnit reflects on a class to find tests and then runs those tests is controlled by a *test runner*. The runner used for a class can be configured with the `@RunWith`

annotation.² JUnit provides a small library of test runners. For example, the Parameterized test runner lets you write data-driven tests in which the same test methods are run for many different data values returned from a static method.

As we'll see below, the jMock library uses a custom test runner to automatically verify mock objects at the end of the test, before the test fixture is torn down.

Hamcrest Matchers and `assertThat()`

Hamcrest is a framework for writing declarative match criteria. While not a testing framework itself, Hamcrest is used by several testing frameworks, including JUnit, jMock, and WindowLicker, which we use in the example in Part III.

A Hamcrest *matcher* reports whether a given object matches some criteria, can describe its criteria, and can describe why an object does not meet its criteria. For example, this code creates matchers for strings that contain a given substring and uses them to make some assertions:

```
String s = "yes we have no bananas today";

Matcher<String> containsBananas = new StringContains("bananas");
Matcher<String> containsMangoes = new StringContains("mangoes");

assertTrue(containsBananas.matches(s));
assertFalse(containsMangoes.matches(s));
```

Matchers are not usually instantiated directly. Instead, Hamcrest provides static factory methods for all of its matchers to make the code that creates matchers more readable. For example:

```
assertTrue(containsString("bananas").matches(s));
assertFalse(containsString("mangoes").matches(s));
```

In practice, however, we use matchers in combination with JUnit's `assertThat()`, which uses matcher's self-describing features to make clear exactly what went wrong when an assertion fails.³ We can rewrite the assertions as:

```
assertThat(s, containsString("bananas"));
assertThat(s, not(containsString("mangoes"));
```

The second assertion demonstrates one of Hamcrest's most useful features: defining new criteria by combining existing matchers. The `not()` method is a factory function that creates a matcher that reverses the sense of any matcher passed to it. Matchers are designed so that when they're combined, both the code and the failure messages are self-explanatory. For example, if we change the second assertion to fail:

-
2. By the time of publication, JUnit will also have a `Rule` annotation for fields to support objects that can “intercept” the lifecycle of a test run.
 3. The `assertThat()` method was introduced in JUnit 4.5.

```
assertThat(s, not(containsString("bananas")));
```

the failure report is:

```
java.lang.AssertionError:  
Expected: not a string containing "bananas"  
got: "Yes, we have no bananas"
```

Instead of writing code to explicitly check a condition and to generate an informative error message, we can pass a matcher expression to `assertThat()` and let it do the work.

Hamcrest is also user-extensible. If we need to check a specific condition, we can write a new matcher by implementing the `Matcher` interface and an appropriately-named factory method, and the result will combine seamlessly with the existing matcher expressions. We describe how to write custom Hamcrest matchers in Appendix B.

jMock2: Mock Objects

jMock2 plugs into JUnit (and other test frameworks) providing support for the mock objects testing style introduced in Chapter 2. jMock creates mock objects dynamically, so you don't have to write your own implementations of the types you want to mock. It also provides a high-level API for specifying how the object under test should invoke the mock objects it interacts with, and how the mock objects will behave in response.



Understanding jMock

jMock is designed to make the expectation descriptions as clear as possible. We used some unusual Java coding practices to do so, which can appear surprising at first. jMock's design was motivated by the ideas presented in this book, backed by many years of experience in real projects. If the examples don't make sense to you, there's more description in Appendix A and at www.jmock.org. We (of course) believe that it's worth suspending your judgment until you've had a chance to work through some of the examples.

The core concepts of the jMock API are the *mockery*, *mock objects*, and *expectations*. A mockery represents the context of the object under test, its neighboring objects; mock objects stand in for the real neighbors of the object under test while the test runs; and expectations describe how the object under test should invoke its neighbors during the test.

An example will show how these fit together. This test asserts that an `AuctionMessageTranslator` will parse a given message text to generate an `auctionClosed()` event. For now, just concentrate on the structure; the test will turn up again in context in Chapter 12.

```

@RunWith(JMock.class) ❶
public class AuctionMessageTranslatorTest {
    private final Mockery context = new JUnit4Mockery(); ❷
    private final AuctionEventListener listener =
        context.mock(AuctionEventListener.class); ❸
    private final AuctionMessageTranslator translator =
        new AuctionMessageTranslator(listener); ❹

    @Test public void
    notifiesAuctionClosedWhenCloseMessageReceived() {
        Message message = new Message();
        message.setBody("SOLVersion: 1.1; Event: CLOSE;"); ❺

        context.checking(new Expectations() {{ ❻
            oneOf(listener).auctionClosed(); ❼
        }});

        translator.processMessage(UNUSED_CHAT, message); ❽
    } ❾
}

```

- ❶ The `@RunWith(JMock.class)` annotation tells JUnit to use the jMock test runner, which automatically calls the mockery at the end of the test to check that all mock objects have been invoked as expected.
- ❷ The test creates the Mockery. Since this is a JUnit 4 test, it creates a `JUnit4Mockery` which throws the right type of exception to report test failures to JUnit 4. By convention, jMock tests hold the mockery in a field named `context`, because it represents the context of the object under test.
- ❸ The test uses the mockery to create a mock `AuctionEventListener` that will stand in for a real listener implementation during this test.
- ❹ The test instantiates the object under test, an `AuctionMessageTranslator`, passing the mock listener to its constructor. The `AuctionMessageTranslator` does not distinguish between a real and a mock listener: It communicates through the `AuctionEventListener` interface and does not care how that interface is implemented.
- ❺ The test sets up further objects that will be used in the test.
- ❻ The test then tells the mockery how the translator should invoke its neighbors during the test by defining a block of expectations. The Java syntax we use to do this is obscure, so if you can bear with us for now we explain it in more detail in Appendix A.
- ❼ This is the significant line in the test, its one expectation. It says that, during the action, we expect the listener's `auctionClosed()` method to be called exactly once. Our definition of success is that the translator will notify its

listener that an `auctionClosed()` event has happened whenever it receives a raw `Close` message.

- ⑧ This is the call to the object under test, the outside event that triggers the behavior we want to test. It passes a raw `Close` message to the translator which, the test says, should make the translator call `auctionClosed()` once on the listener. The mockery will check that the mock objects are invoked as expected while the test runs and fail the test immediately if they are invoked unexpectedly.
- ⑨ Note that the test does not require any assertions. This is quite common in mock object tests.

Expectations

The example above specifies one very simple expectation. jMock's expectation API is very expressive. It lets you precisely specify:

- The minimum and maximum number of times an invocation is expected;
- Whether an invocation is expected (the test should fail if it is not received) or merely allowed to happen (the test should pass if it is not received);
- The parameter values, either given literally or constrained by Hamcrest matchers;
- The ordering constraints with respect to other expectations; and,
- What should happen when the method is invoked—a value to return, an exception to throw, or any other behavior.

An expectation block is designed to stand out from the test code that surrounds it, making an obvious separation between the code that *describes* how neighboring objects should be invoked and the code that actually *invokes* objects and tests the results. The code within an expectation block acts as a little declarative language that describes the expectations; we'll return to this idea in "Building Up to Higher-Level Programming" (page 65).

There's more to the jMock API which we don't have space for in this chapter; we'll describe more of its features in examples in the rest of the book, and there's a summary in Appendix A. What really matters, however, is not the implementation we happened to come up with, but its underlying concepts and motivations. We will do our best to make them clear.

This page intentionally left blank

A Comprehensive Guide Based on Kent C. Dodds' Testing Philosophy

Introduction

Testing is fundamentally about confidence - the confidence that our software will work when users interact with it ^{[1][2]}. Kent C. Dodds, the creator of Testing Library and a prominent voice in the JavaScript testing community, has developed a comprehensive mental framework for understanding how to approach testing effectively. His philosophy revolves around a simple yet profound principle: "The more your tests resemble the way your software is used, the more confidence they can give you" ^{[3][4][5]}.

This comprehensive guide explores the mental models that Kent C. Dodds has developed for testing, drawing from his extensive blog posts and teachings to create a practical framework for understanding when, what, and how to test software applications.

The Foundation: Why We Test

Before diving into specific testing strategies, it's crucial to understand the fundamental purpose of testing ^[2]. Kent C. Dodds emphasizes that we write tests to be confident that our application will work when users interact with it ^[2]. This confidence-driven approach shapes every decision in the testing process.

The primary motivation for testing is not to achieve 100% code coverage or to satisfy organizational requirements, but to prevent bugs from reaching production ^[1]. As Dodds notes, "Much better to catch a bug locally from the tests than getting a call at 2:00 in the morning and fix it then" ^[1]. This practical perspective helps developers understand that testing is an investment in both immediate productivity and long-term maintainability.

The Testing Trophy: A Mental Model for Test Distribution

Kent C. Dodds revolutionized testing strategy with his concept of the Testing Trophy, which challenges the traditional Testing Pyramid approach ^[6]. The Testing Trophy represents four distinct levels of testing, each with its own purpose and trade-offs:

Static Analysis

At the foundation of the trophy sits static analysis . This includes tools like TypeScript, ESLint, and Prettier that catch typos and type errors as code is written . Static analysis provides immediate feedback and prevents entire categories of bugs without requiring any test execution ^[1].

Unit Testing

Unit tests verify that individual, isolated parts work as expected . However, Dodds emphasizes that unit tests should focus on pure functions and complex business logic rather than testing every small piece of code ^[7]. The key insight is that unit tests are most valuable when they test logic that would be difficult to cover thoroughly at higher levels .

Integration Testing

Integration tests verify that several units work together in harmony . This is where Dodds places the most emphasis, as integration tests strike the optimal balance between confidence and cost ^[1]. Integration tests mock as little as possible, typically only mocking network requests and animation components .

End-to-End Testing

At the top of the trophy, end-to-end tests behave like a user interacting with the complete application . These tests provide the highest confidence but are also the most expensive to write and maintain . They should focus on the most critical user journeys and happy paths ^[2].

The Confidence Coefficient: Understanding Test Value

The concept of the "confidence coefficient" is central to Dodds' testing philosophy . This represents the relative confidence that each test level provides. As you move up the testing trophy, the confidence coefficient increases, but so do the costs and complexity .

Static analysis can catch simple syntax errors but cannot verify business logic . Unit tests can verify that individual functions work correctly but cannot ensure that components integrate properly . Integration tests can verify that multiple components work together but may miss issues that only appear in a full production environment . End-to-end tests provide the highest confidence but are slowest to run and most expensive to maintain .

Understanding this trade-off helps developers make informed decisions about where to invest their testing efforts . The goal is not to achieve perfect coverage at every level, but to find the optimal distribution that maximizes confidence while minimizing cost ^[1].

Mental Model for Determining What to Test

One of the most challenging aspects of testing is determining what to test ^[2]. Kent C. Dodds provides a clear mental framework for making these decisions:

Think Use Cases, Not Code Coverage

The fundamental shift in thinking that Dodds advocates is to focus on use cases rather than code coverage [2]. He states: "Think less about the code you are testing and more about the use cases that code supports" [2]. This approach helps prevent the trap of testing implementation details, which leads to brittle tests that break during refactoring [3].

Use case coverage is more valuable than code coverage because it focuses on what users actually need the software to do [2]. When examining untested code, the question should not be "how do I test this function?" but rather "what use cases are these lines of code supporting, and what tests can I add to support those use cases?" [2].

Prioritizing Test Scenarios

When deciding where to start testing in a large application, Dodds recommends asking: "What part of this app would make me most upset if it were broken?" [2]. This user-focused approach helps prioritize testing efforts on the most critical functionality [2].

The process involves:

1. Identifying features that would cause the most damage if broken [2]
2. Creating a prioritized list with team input [2]
3. Starting with end-to-end tests for the most critical happy paths [2]
4. Filling in gaps with integration and unit tests as needed [2]

The TDD Mental Model: Red-Green-Refactor

Test-Driven Development (TDD) represents a specific mental model for writing code [7]. Kent C. Dodds describes TDD as a three-step process often called the "red, green, refactor cycle" [7]:

Red Phase

Write a test for functionality that doesn't exist yet [7]. This test should fail because there's no implementation [7]. The red phase ensures clear understanding of what needs to be built [7].

Green Phase

Implement just enough code to make the test pass [7]. The focus is on making the test green, not on writing perfect code [7]. This phase emphasizes getting to a working state quickly [7].

Refactor Phase

Improve the code without changing its behavior ^[2]. With tests in place, refactoring becomes safe because any breaking changes will be caught immediately ^[2].

When to Apply TDD

Dodds takes a pragmatic approach to TDD, applying it in specific situations where it provides clear benefits ^[2]:

Bug Fixing: When reproducing a bug with a test before fixing it, TDD provides confidence that the bug is understood and actually fixed ^[2].

Pure Functions: Complex utility functions with well-defined inputs and outputs are ideal candidates for TDD ^[2].

Well-Defined User Interfaces: When using Testing Library, TDD becomes practical for UI development because tests can focus on user interactions rather than implementation details ^[2].

Avoiding Implementation Details: A Critical Mental Model

One of the most important mental models in Kent C. Dodds' testing philosophy is understanding and avoiding implementation details ^[3]. Implementation details are "things which users of your code will not typically use, see, or even know about" ^[3].

The Problems with Testing Implementation Details

Testing implementation details creates two major problems ^[3]:

False Negatives: Tests break when refactoring code, even though the functionality remains correct ^[3]. This leads to frustrating maintenance overhead and reduces confidence in the test suite ^[3].

False Positives: Tests pass even when the application is broken, because they're testing internal mechanics rather than user-observable behavior ^[3].

Identifying Implementation Details

To identify implementation details, consider who the users of the code are ^[3]. For React components, there are typically two users: end users who interact with the rendered output, and developers who pass props to the component ^[3].

Implementation details include:

- Internal component state ^[3]
- Private methods ^[3]
- Lifecycle methods ^[3]
- Component instance methods ^[3]

Observable behavior includes:

- Rendered output ^[3]
- Response to user interactions ^[3]
- Callbacks triggered by user actions ^[3]
- Side effects visible to users ^[3]

The Mental Model for Integration Testing

Integration testing represents the sweet spot in Kent C. Dodds' testing philosophy ^[1]. The mental model for integration testing focuses on testing multiple units working together while mocking as little as possible ^[1].

Key Principles of Integration Testing

Minimal Mocking: Only mock external dependencies that are impractical to use in tests, such as network requests or payment processing ^[1]. Everything else should use real implementations .

Real User Interactions: Integration tests should simulate how users actually interact with the application . This means using Testing Library's user interaction utilities rather than calling component methods directly .

Provider Testing: Integration tests should render components with all the providers they would have in the real application . This ensures that context, routing, and other app-level concerns are properly tested .

Integration Testing Trade-offs

Integration tests strike a balance between confidence and practicality ^[1]. They provide more confidence than unit tests because they test real integrations, but they're faster and more reliable than end-to-end tests ^[1]. This makes them the ideal foundation for most test suites ^[1].

Mental Models for Different Test Types

Static Analysis Mental Model

Static analysis tools should be configured to catch errors automatically without requiring developer intervention . The mental model is to "fail fast" - catch problems as early as possible in the development process .

TypeScript and ESLint configurations should be strict enough to prevent common errors but not so restrictive that they impede development ^[1]. The goal is to eliminate entire categories of bugs before any code execution .

Unit Testing Mental Model

Unit tests should focus on pure functions and complex business logic . The mental model is to test the "what" not the "how" - focus on inputs and outputs rather than internal implementation ^[7].

For React components, unit tests should verify that components render correctly with different props and respond appropriately to user interactions . However, they should not test implementation details like state management or lifecycle methods ^[3].

End-to-End Testing Mental Model

End-to-end tests should behave exactly like a real user . The mental model is to automate the manual testing process that would otherwise be performed by QA or users .

E2E tests should focus on critical user journeys and happy paths ^[2]. They should not attempt to cover every edge case, as this would make the test suite too slow and expensive . Instead, edge cases should be covered by integration or unit tests ^[2].

The Mental Model for Test Maintenance

Test maintenance is a crucial aspect of Kent C. Dodds' testing philosophy ^[3]. The mental model is that good tests should rarely need to change when code is refactored ^[3].

Characteristics of Maintainable Tests

Implementation Detail Free: Tests that avoid implementation details are naturally more maintainable ^[3]. They continue to work even when internal code structure changes ^[3].

User-Focused: Tests that focus on user interactions and observable behavior are more stable over time ^[3]. User requirements change less frequently than implementation details ^[3].

Appropriate Abstraction Level: Each test should be written at the appropriate level of abstraction . Unit tests should test units, integration tests should test integrations, and E2E tests should test complete user journeys .

Building Confidence Through Testing Strategy

The overarching mental model in Kent C. Dodds' approach is building confidence in software ^{[1][2]}. This confidence comes from having tests that closely resemble how the software is actually used ^{[3][4][5]}.

Confidence-Driven Test Selection

When choosing what to test, the question should always be: "What would give me the most confidence that this feature works correctly?" ^[1]. This might mean writing an integration test for complex component interactions, or a unit test for intricate business logic .

Balancing Confidence and Cost

The mental model recognizes that perfect confidence is impossible and infinitely expensive . The goal is to find the optimal balance where the test suite provides sufficient confidence while remaining maintainable and fast enough for continuous integration ^[1].

Practical Application of Testing Mental Models

Starting a Testing Strategy

When implementing testing in a new project or improving an existing test suite, Dodds recommends starting with the most critical functionality ^[2]. The mental model is to:

1. Identify the most important user journeys ^[2]
2. Write end-to-end tests for these critical paths ^[2]
3. Add integration tests for important component interactions ^[2]
4. Include unit tests for complex business logic ^[2]
5. Configure static analysis tools to catch common errors

Evolving Test Suites

As applications grow and change, test suites should evolve to maintain confidence while controlling costs ^[1]. The mental model is to regularly evaluate whether tests are still providing value and refactor or remove tests that have become obsolete ^[3].

Team Collaboration

Testing is most effective when the entire team understands and buys into the testing philosophy ^[2]. The mental model emphasizes that testing is not just a developer responsibility but a team effort that includes product managers, designers, and QA professionals ^[2].

Advanced Testing Concepts

Testing User Experience

Beyond functional testing, Kent C. Dodds emphasizes testing the user experience ^[4]. This includes accessibility testing, performance testing, and usability testing ^[8]. The mental model is that tests should verify not just that features work, but that they work well for all users ^[4].

Continuous Integration Integration

Tests should be integrated into the continuous integration pipeline in a way that provides fast feedback while maintaining confidence ^[1]. The mental model is to run fast tests (static analysis, unit tests) first, followed by slower tests (integration, E2E) .

Test Data Management

Effective testing requires good test data management . The mental model is to use realistic test data that exercises the full range of application behavior while being maintainable and deterministic .

Conclusion

Kent C. Dodds' testing philosophy provides a comprehensive mental framework for approaching software testing ^{[1][2][3][4][5]}. The key insights include:

1. **Confidence is the primary goal** - tests should provide confidence that the software works for users ^{[1][2]}
2. **Resemble actual usage** - tests should mirror how the software is actually used ^{[3][4][5]}
3. **Avoid implementation details** - focus on user-observable behavior rather than internal mechanics ^[3]

4. **Balance confidence and cost** - find the optimal distribution of test types for maximum return on investment
5. **Think in use cases** - focus on what users need the software to do rather than achieving code coverage metrics [2]

By internalizing these mental models, development teams can create test suites that provide genuine confidence in their software while remaining maintainable and cost-effective [1][2][3]. The ultimate goal is not perfect testing, but confident shipping - knowing that when code reaches users, it will work as intended [1][2].

This framework represents a mature, practical approach to testing that has been proven in real-world applications and continues to evolve with modern development practices [1][2][3][4][5]. By adopting these mental models, teams can move beyond testing as a chore to testing as a valuable engineering practice that enables confident, frequent delivery of high-quality software.

1. <https://kentcdodds.com/blog/how-to-know-what-to-test>
2. <https://testdriven.io/test-driven-development/>
3. <https://symflower.com/en/company/blog/2023/what-is-the-testing-trophy/>
4. <https://kentcdodds.com/blog/the-time-i-messed-up>
5. <https://courses.cs.washington.edu/courses/cse403/16au/lectures/L07.pdf>
6. <https://reflect.run/articles/introduction-to-react-testing-library/>
7. <https://blog.scottlogic.com/2023/11/06/testing-with-intent-a-technical-view.html>
8. [https://www.reddit.com/r/developersIndia/comments/1ily2uf/what is a good resource to learn test driven/](https://www.reddit.com/r/developersIndia/comments/1ily2uf/what_is_a_good_resource_to_learn_test_driven/)

Invoice Aggregate Class Diagram vs. Pytest Tests

Class Diagram (Invoice Aggregate)

```
@startuml
class Invoice {
    - id: UUID
    - customer: str
    - issue_date: datetime
    - due_date: datetime
    - line_items: List[LineItem]
    - is_paid: bool = False

    + __init__(customer: str, due_days: int)
    + add_line_item(description: str, quantity: int, unit_price: Decimal)
    + remove_line_item(item_id: UUID)
    + calculate_total() -> Decimal
    + mark_paid()
    + get_due_date() -> datetime
}

class LineItem {
    - id: UUID
    - description: str
    - quantity: int
    - unit_price: Decimal

    + __init__(description: str, quantity: int, unit_price: Decimal)
    + calculate_subtotal() -> Decimal
}

Invoice "1" *-- "0..*" LineItem
@enduml
```

Pytest Test Suite

```
import pytest
from decimal import Decimal
```

```

from datetime import datetime, timedelta
from uuid import UUID
from invoice import Invoice, LineItem

def test_invoice_initialization():
    """Test invoice creation with due date calculation"""
    invoice = Invoice("Acme Corp", due_days=30)
    assert invoice.customer == "Acme Corp"
    assert invoice.get_due_date() == datetime.now().date() + timedelta(days=30)
    assert invoice.calculate_total() == Decimal("0.00")
    assert not invoice.is_paid

def test_add_line_item_and_total_calculation():
    """Test adding items and total calculation"""
    invoice = Invoice("Beta LLC", 15)
    item1 = LineItem("Widget A", 2, Decimal("10.50"))
    item2 = LineItem("Gadget B", 1, Decimal("25.00"))

    invoice.add_line_item(item1)
    invoice.add_line_item(item2)

    assert len(invoice.line_items) == 2
    assert invoice.calculate_total() == Decimal("46.00")  # (2*10.50) + 25.00

def test_remove_line_item():
    """Test item removal and total update"""
    invoice = Invoice("Gamma Inc", 10)
    item = LineItem("Service Fee", 1, Decimal("99.99"))
    invoice.add_line_item(item)
    item_id = item.id

    invoice.remove_line_item(item_id)
    assert len(invoice.line_items) == 0
    assert invoice.calculate_total() == Decimal("0.00")

def test_mark_paid():
    """Test payment status update"""
    invoice = Invoice("Delta Co", 7)
    invoice.mark_paid()
    assert invoice.is_paid is True

```

```
def test_line_item_subtotal_calculation():
    """Test line item price calculation"""
    item = LineItem("Bulk Item", 100, Decimal("0.75"))
    assert item.calculate_subtotal() == Decimal("75.00")
```

Test Coverage Checklist

For each public method in your UML, write at least one behavioural test:

Method Signature	Example Test Focus
<code>__init__(customer, due_days)</code>	Initial state validation
<code>add_line_item(...)</code>	Item addition + total recalculation
<code>remove_line_item(item_id)</code>	Item removal + total update
<code>calculate_total() -> Decimal</code>	Multi-item calculation accuracy
<code>mark_paid()</code>	Payment status change
<code>get_due_date() -> datetime</code>	Due date calculation logic
<code>LineItem.calculate_subtotal()</code>	Quantity×price calculation

Key Testing Principles Demonstrated

- 1. **State Verification:** Checks object state after operations (e.g., `assert len(invoice.line_items) == 2`)
- 2. **Boundary Conditions:** Tests zero/negative totals when all items removed
- 3. **Behavior Isolation:** Tests `LineItem` independently from `Invoice`
- 4. **Temporal Logic:** Validates date calculations relative to current date
- 5. **Precision Handling:** Uses `Decimal` for financial calculations